



Do not Mix Models

Revisiting Generic Transforms for Committing Authenticated Encryption

Venkatesh (220109)

Anas (220654)

Modern Cryptology Course Presentation

May 3, 2026

Based on: Minematsu & Inoue, Do not Mix Models (IACR ePrint)

- Background and Motivation
- The Transforms Under Attack
- Instantiation Attacks
- Provably Secure Fixes
- Discussion and Conclusion

Authenticated Encryption - A Quick Recap

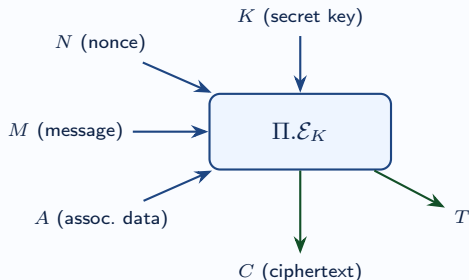
AE combines **confidentiality** and **integrity** into a single primitive.

- ▶ Inputs: key K , nonce N , associated data A , message M
- ▶ Output: ciphertext C (which includes a tag T)
- ▶ Security goals:
 - ▷ **PRIV**: (C, T) indistinguishable from random
 - ▷ **AUTH**: adversary cannot forge a valid (C, T)

Standard Examples

AES-GCM · ChaCha20-Poly1305 · Ascon (NIST LWC 2023)

The key K is traditionally treated as **secret** in security definitions - an adversary cannot choose or observe it.



Decryption $\Pi.\mathcal{D}_K$ reverses this and returns $\perp$ if the tag fails.

Committing Security: Beyond Privacy and Authenticity

Traditional AE treats the key as **secret**. But what if an adversary tries to find **two different inputs** that produce the **same ciphertext**?

Context-Committing Security (CMT-4)

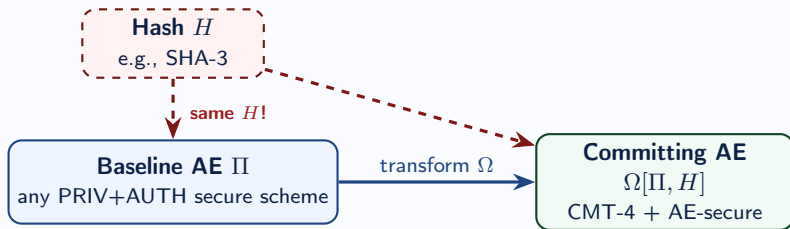
Hard to find $(K, N, A, M) \neq (K', N', A', M')$ such that:

$$\Pi.\mathcal{E}(K, N, A, M) = \Pi.\mathcal{E}(K', N', A', M')$$

Why does this matter?

- ▶ **Partition oracle attacks:** exploit key-commitment failures to test key guesses
- ▶ **Invisible salamanders:** craft files that decrypt to different content under different keys
- ▶ Practical attacks shown against AES-GCM, AES-GCM-SIV, XSalsa20-Poly1305, etc.

Generic Transforms



Design goal of a generic transform

- ▶ **Goal:** convert *any* AE scheme Π into a committing one
- ▶ Transform accesses Π in a **black-box** way - only uses $\Pi.\mathcal{E}$ and $\Pi.\mathcal{D}$
- ▶ Adds a hash function H to bind inputs to the ciphertext

The Mixed Security Model: A Structural Gap

What the security proof assumes

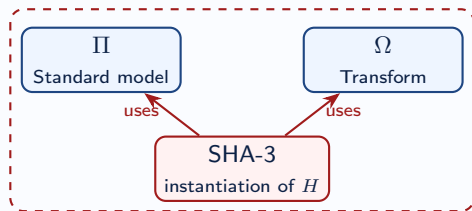
- ▶ Π is AE-secure in the **standard model** (PRF, PRP, collision resistance, etc.)
- ▶ H behaves as a **random oracle** (an ideal, public, truly random function)

What happens in reality

When the random oracle H is instantiated with a concrete hash such as SHA-3:

- ▶ Π *can* internally use the same SHA-3
- ▶ H is no longer independent of Π 's internals
- ▶ The RO proof **does not account** for Π knowing H 's algorithm

shared instantiation



The adversary can craft Π *using the same hash* so that Π 's internals interfere with Ω 's use of H .

Contributions of This Paper

(1) Practical attacks on existing transforms

- ▶ Targets: **AEaH, EtH, AEtH, chaSIV** (Chen–Karadžić, Eurocrypt 2025)
- ▶ Targets: **PACT, comPACT** (Bhattacharjee et al., ePrint 2024)
- ▶ Attack types: privacy, authenticity, plaintext recovery, *key recovery*
- ▶ Baseline AEs are **provably secure** and structurally natural

(2) Classification framework

- ▶ **Uninstantiable**: broken for *any* H
- ▶ **Conditionally uninstantiable**: broken for *some* H

(3) Standard-model repairs via Key Separation

- ▶ Use a PRG to derive two independent keys
- ▶ Minimal algorithmic change; cacheable cost
- ▶ Entirely **standard-model** proofs (PRG, PRF, collision resistance)

New methods: AEaH-KS and AEtH-KS are the *first* transforms that are simultaneously

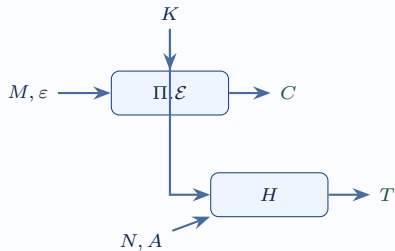
- (1) strictly generic
- (2) CMT-4 secure
- (3) free from nonce-based rekeying
- (4) entirely standard-model secure

The Transforms Under Attack

CK25 Basic Transforms: AEaH, EtH, HtAE

All three proposed by Chen & Karadžić at **Eurocrypt 2025**. Security proofs assume H is a random oracle.

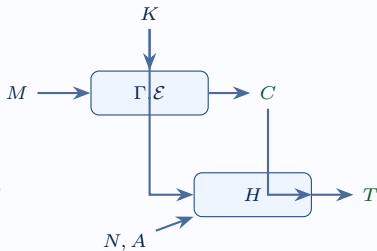
AE and Hash (AEaH)



$$T = H(K, N, A)$$

Tag fully from hash; ciphertext from Π

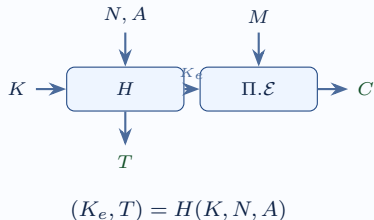
Enc-then-Hash (EtH)



$$T = H(K, N, A, C)$$

Γ is plain (unauthenticated) encryption

Hash-then-AE (HtAE)



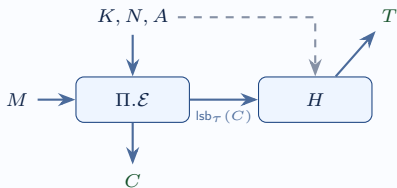
Hash derives both encryption key and tag

- AEaH and EtH achieve PRIV+AUTH+CMT-4 in nonce-respecting setting.
- HtAE derives an encryption key from the nonce/context, so it has a rekeying flavor.

CK25 Advanced Transforms: AEtH and chaSIV

Both achieve misuse-resistant security (MRAE) with CMT-4.

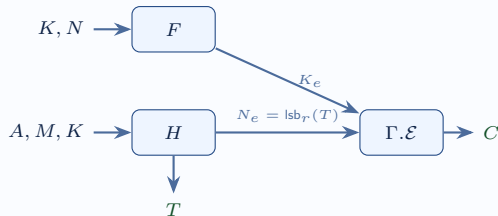
AE-then-Hash (AEtH)



$$T = H(K, N, A, \text{lsb}_\tau(C))$$

MRAE-preserving: inherits MRAE from Π

Committing hash-based SIV (chaSIV)



$$K_e = F(K, N), \quad T = H(K, N, A, M)$$

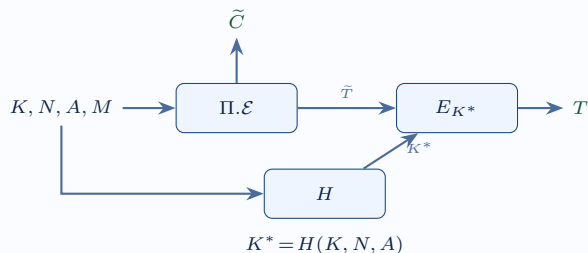
MRAE-lifting: upgrades a plain encryption Γ

Already a mixed model internally

chaSIV uses both F (standard-model PRF) and H (RO) simultaneously - making it especially vulnerable to instantiation issues.

BBD24 Transforms: PACT and comPACT

Core idea: *encrypt the baseline AE's tag* using a block cipher keyed by a hash of the context.



Key design features

- ▶ Black-box access to Π : no need to expose internal tag
- ▶ E is an ideal cipher keyed by H -output
- ▶ **comPACT**: replaces AD to Π with ε , slightly faster

Security assumptions

- ▶ H is collision resistant (not just RO)
 - ▶ E is an ideal cipher
- ⇒ Still a **mixed model**

Attack Strategy: How the Mixed Model Breaks

Key Observation

When H is instantiated (e.g., with SHA-3), the baseline AE Π can **use** H internally. The adversary constructs a Π that is **provably secure** on its own, but **leaks information** through the shared H when composed with the transform.

1. Design Π using H
as a PRF/PRG



2. Π is AE-secure
(standard model)



3. $\Omega[\Pi, H]$ is
completely broken

Implication

A malicious party could embed a **backdoor** into a protocol using black-box AE combined with such transforms. The standalone AE passes all security tests, but the composed scheme is broken.

Privacy Attack on AEaH

Setup: Let Π be an Enc-then-MAC AE using H . For an mt -bit plaintext $M = (M[1] \parallel \dots \parallel M[m])$, it encrypts to $C = (\tilde{C}, \tilde{T})$ where $\tilde{C} = (\tilde{C}[1] \parallel \dots \parallel \tilde{C}[m])$. The encryption uses H in counter mode:

$$\tilde{C}[i] = H(K, N, \langle i \rangle) \oplus M[i], \quad \tilde{T} = \text{MAC}_K(N, A, \tilde{C})$$

This Π is **AE-secure** if H_K is a PRF (standard Enc-then-MAC result).

Attack (choose $A = \langle 1 \rangle$):

1. Encrypt $(N, A=\langle 1 \rangle, M)$; receive output (C, T) where $T = H(K, N, A) = H(K, N, \langle 1 \rangle)$
2. From C : extract $\tilde{C}[1] = H(K, N, \langle 1 \rangle) \oplus M[1]$
3. Check: $\tilde{C}[1] \oplus T \stackrel{?}{=} M[1]$

Since $\tilde{C}[1] = H(K, N, \langle 1 \rangle) \oplus M[1]$ and $T = H(K, N, \langle 1 \rangle)$:

$$\tilde{C}[1] \oplus T = M[1] \quad \text{always hold!}$$

Consequences

- Distinguishing advantage ≈ 1 (vs. $\approx 1/2^t$ in ideal world)
 - **Plaintext recovery:** setting $A = \langle j \rangle$ recovers $M[j]$ for any j
 - Works for **any** instantiation of H
- $\Rightarrow$ AEaH is **uninstantiable**

Authenticity Attack against AEaH

Setup: Instantiate Π as a GCM variant combining Γ and GHASH. Let the nonce space be $\mathcal{N} = \{0, 1\}^n \setminus \{\text{cst}_1, \text{cst}_2\}$. Define $\Pi.\mathcal{E}(K, N, A, M) = C = (\tilde{C}, \tilde{T})$, where:

$$L_i = H(K, \text{cst}_i, \varepsilon) \text{ for } i = 1, 2, \quad \tilde{C} = \Gamma.\mathcal{E}(L_1, N, M), \\ \tilde{T} = \text{GHASH}(L_2, (A, \tilde{C})) \oplus H(K, N, A).$$

Attack procedure (set $A = \varepsilon$):

1. Encrypt (N, ε, M) , receive (C, T) where $C = (\tilde{C}, \tilde{T})$, $T = H(K, N, \varepsilon)$
2. Compute: $\tilde{T} \oplus T = \text{GHASH}(L_2, (\varepsilon, \tilde{C})) \rightarrow$ **recover** L_2
3. Craft $\tilde{C}' \neq \tilde{C}$ with $\text{GHASH}(L_2, (\varepsilon, \tilde{C}')) = \text{GHASH}(L_2, (\varepsilon, \tilde{C}))$
4. Submit forged decryption query — **accepted with probability 1**

Universal forgery

Once L_2 is known, an attacker can forge a valid ciphertext for *any* plaintext.

Privacy and Authenticity Attacks against EtH

Setup for Privacy Attack: Instantiate Γ using an H -based CTR mode:

$$C[i] = \text{msb}_{|M[i]|}(H(K, \text{cst}, N, \langle i-1 \rangle)) \oplus M[i]$$

While this Γ is PRIV-secure, its argument structure misaligns dangerously with the EtH transform tag $T = H(K, N, A, C)$.

Privacy Attack Procedure:

1. Query 1: Encrypt with $N = \text{cst}$. The resulting tag is $T = H(K, \text{cst}, A, C)$.
2. Query 2: Encrypt a new message with $N' = A$. The keystream block for index j (where $\langle j-1 \rangle = C$) becomes exactly $H(K, \text{cst}, A, C)$.
3. **Exploit:** Since the tag T from Query 1 equals the keystream block for Query 2, we can perfectly predict the ciphertext block $C'[j] = T \oplus M'[j]$. Distinguishing advantage ≈ 1 .

Authenticity Attack: Similar to AEaH, we can extract the internal keystream to forge tags:

1. Encrypt (N, A, M) and extract the keystream $H(K, N, \text{cst}, \langle i \rangle) = M[i] \oplus C[i]$.
2. Forge a new ciphertext by submitting $(N, A' = \text{cst}, C' = \langle i \rangle, T' = M[i] \oplus C[i])$.
3. **Result:** Accepted with probability 1 because the extracted keystream matches the required tag equation $T' = H(K, N, A', C')$.

Privacy Attack against AEtH and chaSIV

AEtH attack:

Use SIV-like Π with H -based encryption:

$$C[i] = M[i] \oplus H(K, N, A, V \oplus \langle i-1 \rangle_\tau)$$

where $V = F(K, N, A, M)$.

After AEtH transform, $T = H(K, N, A, V)$.

When $i = 1$: $V \oplus \langle 0 \rangle_\tau = V$, so

$$C[1] \oplus M[1] = H(K, N, A, V) = T$$

$\Rightarrow$ **Plaintext recovery** of $M[1]$.

chaSIV attack:

Let $F(K, N) = \text{msb}_\kappa(H(K, N, \text{cst}_1, \text{cst}_2))$

Encrypt $(N, A=\text{cst}_1, M=\text{cst}_2)$:

$$T = H(K, N, \text{cst}_1, \text{cst}_2)$$

$$\Rightarrow K_e = \text{msb}_\kappa(T)$$

Consequence: encryption key K_e is revealed directly from the tag. Complete privacy break.

Note: needs F to use H , hence chaSIV is **conditionally** uninstantiable.

Attacks against PACT and comPACT (BBD24)

Recall: $K^* = H(K, N, A)$, $T = E(K^*, \tilde{T})$

Privacy attack: Use SIV-style Π where $\tilde{C}[i] = M[i] \oplus E(K^*, \tilde{T} \oplus \langle i-1 \rangle_\tau)$.

Since PACT outputs $T = E(K^*, \tilde{T})$:

$$\tilde{C}[1] = M[1] \oplus E(K^*, \tilde{T}) = M[1] \oplus T \quad \Rightarrow \quad M[1] = \tilde{C}[1] \oplus T$$

Authenticity attack: Use GCM-variant Π with GHASH.

PACT computes $T = E(K^*, E^{-1}(K^*, V)) = V$, directly revealing GHASH output.

1. Recover GHASH key L from $T = V = \text{GHASH}(L, (A, \tilde{C}))$
2. Find $\tilde{C}' \neq \tilde{C}$ with same GHASH value $\rightarrow$ forgery accepted

Both PACT and comPACT Are Uninstantiable

Privacy and authenticity broken for **any** instantiation. Attacks extend to comPACT with minor modifications.

Key Recovery attacks via Key Blinding

Some transforms allow **total key recovery** using a simple augmentation of the attack AE schemes.

Technique: Key Blinding (KB)

Replace the counter-mode AE with a key-blinded variant:

$$\tilde{C}[i] = M[i] \oplus H(K, N, \langle i \rangle) \oplus K$$

The extra $\oplus K$ does not affect provable AE security, but converts the privacy attack into key recovery:

$$K = M[1] \oplus \tilde{C}[1] \oplus T$$

Transform	Privacy / Auth broken?	Key recovery?
AEaH	any instantiation	with KB
EtH	any instantiation	with KB
AEtH	any instantiation	with KB
chaSIV	some instantiations	specialized F, Γ
PACT / comPACT	any instantiation	with KB

Summary of Attacks

Scheme	Ref.	Strictly Generic	Model	Priv	Auth	Key Rec.
CTX	CR22	×	Mixed		no attack found	
CTY	BH24	×	Mixed		no attack found	
AEaH	CK25	✓	Mixed	×	×	(×)
EtH	CK25	✓	Mixed	×	×	(×)
AEtH	CK25	✓	Mixed	×	—	(×)
chaSIV	CK25	✓	Mixed	× _e	—	(× _e)
PACT	BBD24	≈✓	Mixed	×	×	(×)
comPACT	BBD24	≈✓	Mixed	×	×	(×)
HtE-UtC/RtC	BH22	✓	Standard	no attack (entirely standard model)		
HtAE	CK25	✓	Mixed	no attack found		

× attack for any instantiation
 ×_e attack for a class of instantiations
 (×) key recovery via KB augmentation

Provably Secure Fixes

Root Cause and the Key Separation Fix

Root Cause

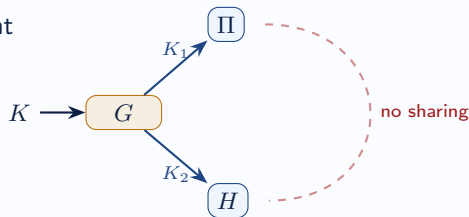
The baseline AE Π can **simulate** the computation of the transform's hash, because they share the same instantiation of H .

Solution - Key Separation (KS):

- ▶ Apply a PRG $G : \mathcal{K} \rightarrow \mathcal{K} \times \mathcal{K}$ to derive two independent keys:

$$(K_1, K_2) \leftarrow G(K)$$

- ▶ K_1 is used for the baseline AE Π
- ▶ K_2 is used for the hash function H
- ▶ **No shared state** between Π and H after separation

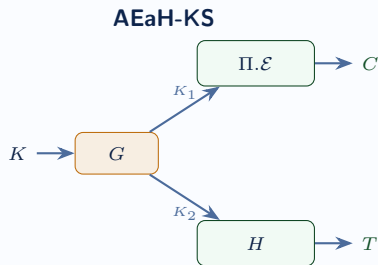


Properties of KS

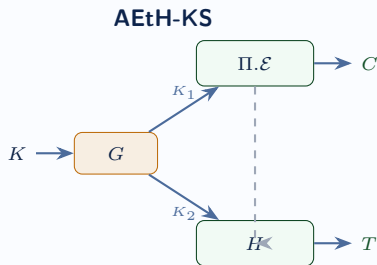
- ▶ Computation of $G(K)$ can be **cached** per key - negligible online cost
- ▶ No nonce-based rekeying needed
- ▶ Entirely **standard-model** assumptions: G is a PRG, H_{K_2} is a CR-PRF

Fixed Transforms: AEaH-KS, AEtH-KS, EtH-KS

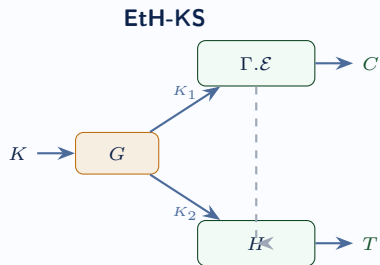
All three apply Key Separation; only the hash input changes per transform.



$$T = H(K_2, N, A)$$



$$T = H(K_2, N, A, \text{lsb}_\tau(C))$$



$$T = H(K_2, N, A, C)$$

Why KS works

$G(K)$ produces (K_1, K_2) that are pseudorandom and *computationally independent*. Even if Π internally uses the same hash algorithm, it uses key K_1 , not K_2 , so it cannot simulate H_{K_2} .

Theorem 1 - AE Security

$$\text{Adv}_{\text{AEaH-KS}}^{\text{priv}}(\mathcal{A}) \leq \text{Adv}_G^{\text{prg}}(\mathcal{A}_G) + \text{Adv}_H^{\text{prf}}(\mathcal{A}_H) + \text{Adv}_{\Pi}^{\text{priv}}(\mathcal{A}_{\Pi})$$

$$\text{Adv}_{\text{AEaH-KS}}^{\text{auth}}(\mathcal{A}) \leq \text{Adv}_G^{\text{prg}} + \text{Adv}_H^{\text{prf}} + \text{Adv}_{\Pi}^{\text{auth}} + \frac{qd}{2^t}$$

Theorem 2 - Committing Security

$$\text{Adv}_{\text{AEaH-KS}}^{\text{cmt}}(\mathcal{A}_{\text{cmt}}) \leq \text{Adv}_H^{\text{cr}}(\mathcal{A}_H) + \text{Adv}_G^{\text{rcr}}(\mathcal{A}_G)$$

Proof technique: Hybrid argument

1. Replace G with uniform random source $\$G$ (cost: $\text{Adv}_G^{\text{prg}}$)
2. Replace H_{K_2} with a URF R_H (cost: $\text{Adv}_H^{\text{prf}}$)
3. T part becomes independent uniform random $\Rightarrow$ reduce to Π 's security

Fixing chaSIV: Nonce-Based Key Separation

chaSIV-NKS: Original chaSIV used $F(K,N)$ for the encryption key and $H(K,N,A,M)$ for the tag. The attack worked when F was built from the same H . We remove N from H 's input, use F for both K_e and K_h . This prevents the old attack where T directly revealed $F(K,N)$.

chaSIV-NKS Encryption

1. $K_e \leftarrow F(K, 0||N)$
2. $K_h \leftarrow F(K, 1||N)$
3. $T \leftarrow H(K_h, A, M)$
4. $N_e \leftarrow \text{lsb}_r(T)$
5. $C \leftarrow \Gamma.\mathcal{E}(K_e, N_e, M)$
6. return (C, T)

Key Differences from Original

- ▶ H no longer takes N as input - prevents the attack where F reuses H
- ▶ PRF F derives two **independent** keys per nonce
- ▶ Domain separation via prefix bit ($0||N$ vs. $1||N$)
- ▶ Roughly preserves original complexity

Security: MR-PRIV and MR-AUTH under PRF assumptions on F and H , PRIV of Γ .

CMT: reduces to CR of H + right-half CR of F .

HtAE Under the Standard Model (HtAE-ST)

HtAE is the **only** CK25 transform where no instantiation attack was found.

Observation

HtAE can be proven secure **without changing the algorithm** - only the assumption on H changes:

- ▶ Original proof: H is a random oracle
- ▶ New proof: H is a **PRF** (keyed by K) and is **right-half collision resistant**

Theorem 9 - HtAE-ST Security

$$\begin{aligned}\text{Adv}_{\text{HtAE-ST}}^{\text{priv}}(\mathcal{A}) &\leq \text{Adv}_H^{\text{prf}} + q_e \cdot \text{Adv}_{\Pi}^{\text{priv}} \\ \text{Adv}_{\text{HtAE-ST}}^{\text{auth}}(\mathcal{A}) &\leq \text{Adv}_H^{\text{prf}} + Q_d \cdot \text{Adv}_{\Pi}^{\text{auth}} + \frac{q_d}{2^t}\end{aligned}$$

CMT security: $\text{Adv}^{\text{cmt}} \leq \text{Adv}_H^{\text{rcr}}$ (right-half collision resistance of H).

Summary of Fixed Transforms

Scheme	Fix	Alg. Change	Nb Rekey	Add. Cost	Model
HtAE-ST	Stronger H assumption	No	Yes	0	Standard
AEaH-KS	Key Separation	Yes	No	$O(\kappa)^\dagger$	Standard
EtH-KS	Key Separation	Yes	No	$O(\kappa)^\dagger$	Standard
AEtH-KS	Key Separation	Yes	No	$O(\kappa)^\dagger$	Standard
chaSIV-NKS	Nonce-based KS	Yes	Yes	0	Standard

† : PRG computation can be cached per key.

Takeaway

All fixes are **lightweight** - the main cost is a single PRG evaluation per key, which can be cached.
No fundamental performance penalty.

Discussion and Conclusion

Why Random IVs Do Not Solve the Problem

A natural alternative: generate a fresh random IV inside the transform and pass it only to H .

This approach fails

- ▶ The random IV must appear in the **ciphertext output** so the receiver can recompute H
- ▶ The adversary *observes the IV* from the ciphertext
- ▶ In a subsequent query, the adversary feeds the IV to Π (e.g., as part of A or N)
- ▶ Π can now internally compute $H(\dots, IV, \dots)$, reproducing the transform's computation

What is needed instead

Independence must be *cryptographic* - derived via a PRG so that even knowledge of K_1 gives no useful information about K_2 , and vice versa.

In other words: fresh randomness is not enough. The baseline AE can always mirror any fresh value, as long as it appears in the public ciphertext. PRG-based key separation is fundamentally different.

CTX and CTY: Why No Attacks Were Found

Despite using a mixed model, CTX and CTY appear to resist instantiation attacks.

Structural reason

CTX/CTY compute:

$$T = H(K, N, A, \tilde{T})$$

where $\tilde{T}$ is the **internal tag** output by Π .

Crucially, $\tilde{T}$ is *consumed* and *replaced* by T in the output. The adversary never sees $\tilde{T}$, so Π cannot create a useful correlation between its internal computation and $H(\tilde{T})$.

Standard-model variants

Applying Key Separation to CTX/CTY yields **CTX-KS** and **CTY-KS**:

- ▶ $(K_1, K_2) \leftarrow G(K)$
- ▶ Π uses K_1 , H uses K_2
- ▶ AE security bound: PRG + PRF + original CTX/CTY bound (with zero primitive queries)

Open Question

Can we prove standard-model security of CTX/CTY **without** changing the algorithm (as done for HtAE-ST)? This remains open.

Limitations of This Work

1. Adversarially crafted baseline AEs

- ▷ The attack II schemes are constructed specifically to exploit the transform - they are not arbitrary deployed AE schemes
- ▷ That said, they use natural algorithmic structure (Enc-then-MAC, counter mode, GCM-like authentication)

2. No attacks on existing deployments

- ▷ Results do not break AES-GCM or ChaCha20-Poly1305 when composed with these transforms using an independent hash
- ▷ The threat model is: a potentially malicious designer of the baseline AE, not an external attacker

3. PACT/comPACT repairs not provided

- ▷ Fixes are given only for the CK25 transforms; BBD24 transforms are left to future work
- ▷ BBD24 transforms are not strictly generic, complicating the repair strategy

4. Only asymptotic analysis

- ▷ No concrete performance benchmarks or implementation measurements are given for the fixed schemes

Open Problems and Future Directions

1. Standard-model security of CTX/CTY without algorithmic changes

- ▷ Can we replicate the HtAE-ST approach for CTX and CTY?

2. Tighter security bounds for fixed transforms

- ▷ Current proofs are based on direct reduction; dedicated analysis may yield tighter bounds

3. Mixed-model pitfalls in other cryptographic settings

- ▷ The mixed-model vulnerability is a general phenomenon - does it affect other areas (e.g., key exchange, signatures)?

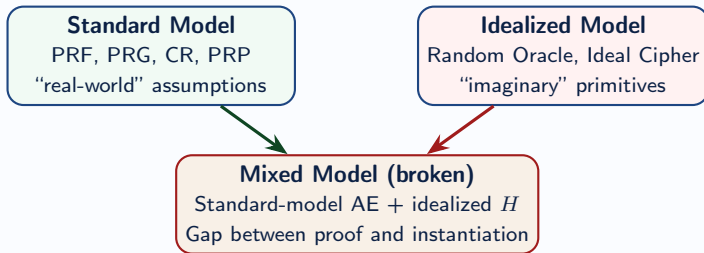
4. Fixing PACT and comPACT

- ▷ BBD24 designers indicated potential revisions; formal standard-model variants are needed

5. Beyond key separation

- ▷ Are there alternative techniques to achieve standard-model security without PRG-based key splitting?

The Broader Lesson: Standard Model vs. Mixed Model



Instantiation-based attacks possible

Takeaway from Minematsu & Inoue

The idealized model is useful for proof sketches, but mixing it with standard-model assumptions creates **blind spots**. Transforms that rely *entirely* on the standard model are inherently immune to instantiation attacks.

Conclusion

1. **Problem identified:** Generic transforms for committing AE that mix standard-model and idealized assumptions are vulnerable when instantiated - even if each component is individually secure.
2. **Concrete attacks:** Privacy, authenticity, plaintext recovery, and key recovery attacks against several state-of-the-art transforms (CK25 and BBD24), using natural and provably secure baseline AE schemes.
3. **Lightweight fixes:** Key Separation (PRG-based key derivation) provides standard-model-secure repairs with essentially zero performance overhead.
4. **New methods:** AEaH-KS and AEtH-KS achieve all four desirable properties simultaneously for the first time: strictly generic, CMT-4 secure, no nonce-based rekeying, entirely standard-model.
5. **Broader message:** *Do not mix models.* When the same primitive appears under two different security assumptions in a composition, the proof may not survive instantiation.

Thank You!

Questions & Discussion

Minematsu & Inoue, 2025

Additional Slides

Formal Definition of Uninstantiability

Definition (Minematsu–Inoue, adapted from paper)

Let $\Omega[*, P]$ be a generic transform and Π an efficient AE scheme. We say $\Omega[\Pi, P]$ is **broken** if any of PRIV/AUTH/CMT is broken with $O(1)$ queries in negligible time.

- ▶ Ω is **uninstantiable** if there exists AE-secure $\Pi[P]$ such that $\Omega[\Pi[P], P]$ is broken for **every** instantiation P .
- ▶ Ω is **conditionally uninstantiable** if there exists AE-secure $\Pi[P]$ such that $\Omega[\Pi[P], P]$ is broken for **some** instantiation P .

Implication

If Ω is (conditionally) uninstantiable, a malicious protocol designer can build a standalone-secure Π that causes $\Omega[\Pi, H]$ to be completely broken in practice.

Contrast with CGH-style uninstantiability

Prior uninstantiable constructions exist because the construction *itself* requires the RO. Here the issue is different: the construction is broken because of *interaction* between two components that share an instantiation.

Nonce-Misuse Resistance (MRAE)

MR-PRIV and MR-AUTH

- ▶ Standard PRIV/AUTH: adversary must use **unique nonces** for each encryption query
- ▶ MR-PRIV/MR-AUTH: adversary may **repeat nonces** across encryption queries (but not repeat the entire query)
- ▶ Captures systems where nonce management may fail in practice (e.g., network protocols with resend)

Key MRAE scheme: SIV

[Rogaway–Shrimpton, Eurocrypt 2006]

- ▶ $V = F(K, N, A, M)$ (synthetic IV via PRF)
- ▶ $C = \text{CTR}_K(V, M)$
- ▶ Output: (C, V) - authenticity via deterministic tag

- ▶ **AEtH** is MRAE-*preserving*: if Π is MRAE-secure, so is $\text{AEtH}[\Pi, H]$
- ▶ **chaSIV** is MRAE-*lifting*: upgrades a nonce-respecting Γ to MRAE
- ▶ Both AEtH-KS and chaSIV-NKS retain these MRAE properties

Proof Sketch: CMT Security of AEaH-KS (Theorem 2)

Goal: $\text{Adv}_{\text{AEaH-KS}}^{\text{cmt}} \leq \text{Adv}_H^{\text{cr}} + \text{Adv}_G^{\text{rcr}}$

Suppose adversary outputs a valid collision: $Z = ((K, N, A, M), (K', N', A', M'), (C, T))$

Let $(K_1, K_2) = G(K)$ and $(K'_1, K'_2) = G(K')$.

Case 1: $K = K'$

- ▶ Then $(K_1, K_2) = (K'_1, K'_2)$
- ▶ If $(N, A) = (N', A')$: tag equality and correctness force $M = M'$ - not a valid collision
- ▶ If $(N, A) \neq (N', A')$: $H(K_2, N, A) = H(K_2, N', A') = T \Rightarrow$ **collision in H**

Case 2: $K \neq K'$

- ▶ If $K_2 = K'_2$: $\Rightarrow$ **right-half collision in G**
- ▶ If $K_2 \neq K'_2$: $H(K_2, N, A) = H(K'_2, N', A') = T \Rightarrow$ **collision in H**

In every case, the adversary must break CR of H or RCR of G .

□

Key Recovery against chaSIV: Full Construction

Construction of F and Γ

Parse $K = (K[0]||K[1])$, $|K[0]| = |K[1]| = \kappa/2$. Let π be a public κ -bit permutation.

$$S[i] = \pi^{-1}(K[i]||0^{\kappa/2}), \quad i \in \{0, 1\}$$

$$F(N, K) = \text{msb}_{\kappa}(V_N) \oplus S[\text{msb}_1(N)] \quad \text{where } V_N = H(K, N, \text{cst}_1, \text{cst}_2)$$

$$\Gamma.\mathcal{E}(K_e, N_e, M) = \text{msb}_{\kappa/2}(\pi(N_e \oplus K_e)) \oplus M$$

Attack with $A = \text{cst}_1$, $M = \text{cst}_2$, $\text{msb}_1(N) = 0$:

$$C \oplus M = \text{msb}_{\kappa/2} \left(\pi \left(\underbrace{\text{msb}_{\kappa}(V_N)}_{N_e} \oplus \underbrace{S[0] \oplus \text{msb}_{\kappa}(V_N)}_{K_e} \right) \right) = \text{msb}_{\kappa/2}(\pi(\pi^{-1}(K[0]||0^{\kappa/2}))) = K[0]$$

Second query with $\text{msb}_1(N') = 1$ recovers $K[1]$. Full key in two queries.

Related Work on Uninstantiability and Committing AE

RO uninstantiability (general):

- ▶ **CGH04**: First uninstantiable RO-model construction
- ▶ **BBP04**: Uninstantiable hybrid encryption
- ▶ **BFM15**: Uninstantiability from iO
- ▶ **Bla06**: Uninstantiable ideal-cipher construction
- ▶ **KRS25**: RO exploitation in Fiat-Shamir

This paper's novelty:

Attacks arise from *mixed-model composition* rather than from the construction requiring the RO intrinsically. A different and practically more relevant pitfall.

Committing AE (previous work):

- ▶ **FOR17**: First formalization of committing AE
- ▶ **GLR17**: Message franking via committing AE
- ▶ **DGRW18**: Invisible salamanders
- ▶ **ADG+22**: Attacks on deployed schemes
- ▶ **LGR21**: Partitioning oracle attacks
- ▶ **BH22**: HtE-UtC-RtC generic transforms (standard model)
- ▶ **MLGR23**: Context discovery attacks on CCM, EAX, SIV
- ▶ **NSS24**: KIVR - dedicated construction for GCM